Módulo 5 — Plataforma (transversal)

1. Propósito

Reúne lo que usan todos los módulos: cómo se arma la API, cómo se validan los datos y se responden los errores, cómo se registran los eventos (observabilidad), el chequeo de salud, la configuración por variables de entorno, los tests y el pipeline de CI/CD. El registro de eventos es el **proceso 5 "Registrar eventos"** de los DFD. No tiene casos de uso propios: da soporte a los CU-01 a CU-17.

2. Dónde está en el código

Archivo	Qué hace
`backend/src/server.ts`	Punto de entrada: lee la configuración y levanta el servidor
`backend/src/app.ts`	Arma la app Express: `helmet`, CORS, JSON de hasta 100 kB, logs HTTP, rutas y manejo de errores
`backend/src/config/env.ts`	**Única** lectura de variables de entorno; falla al arrancar si falta `DATABASE_URL` o `JWT_SECRET`
`backend/src/lib/errors.ts`	`HttpError(status, mensaje, codigo?)`
`backend/src/middleware/errorHandler.ts`	`noEncontrado` (404) y `manejadorErrores` (400 / 4xx / 500)
`backend/src/lib/logger.ts`	Logger `pino` en JSON, con datos sensibles ocultos
`backend/src/routes/health.ts`	`GET /api/health`
`backend/src/lib/prisma.ts`	Cliente único de Prisma
`frontend/src/api/cliente.ts`	Única puerta del frontend a la API (`pedirApi`, `enviar`, manejo de 401)
`backend/vitest.config.ts`	Tests, base de pruebas y umbral de cobertura del 65 %
`.github/workflows/ci-cd.yml`	Pipeline de CI/CD
`docker-compose.yml`	Bases locales de desarrollo (5432) y de tests (5433)

3. Validación y errores

Toda entrada se valida con **zod** al principio de cada ruta (`esquema.parse(req.body)`). Si algo falla, el error viaja solo hasta el manejador central (Express 5 lo hace también en rutas `async`):

Situación	Respuesta
Datos que no cumplen el esquema (`ZodError`)	`400 { error: "Datos inválidos", detalles: [{ campo, mensaje }] }`
JSON mal formado	`400 { error: "El cuerpo de la petición no es un JSON válido" }`

Situación	Respuesta
Regla de negocio, permiso o recurso (`HttpError`)	su código (401, 403, 404, 409, 502, 503) con `{ error, codigo? }`
Ruta inexistente	`404 { error: "Recurso no encontrado", ruta }`
Cualquier otro error	`500 { error: "Error interno del servidor" }`, sin detalles internos; el detalle va al log

Ningún dato inválido llega a la base ni termina en un 500.

4. Observabilidad (proceso 5 “Registrar eventos”)

La API escribe **logs estructurados en JSON** (pino) en la salida estándar; en producción los guarda y muestra **Render Logs**. Cada línea tiene la hora, el nivel, `servicio: "viandassemana-api"` y, en los eventos de negocio, un campo `evento` por el que se puede buscar. `pino-http` agrega una línea por petición con método, ruta, código de respuesta y tiempo.

`evento`	Nivel	Cuándo	Origen
`login_fallido`	warn	Email o contraseña incorrectos	`routes/auth.ts`
`recuperacion_solicitada`	info	Se mandó un código de recuperación	`routes/auth.ts`
`password_restablecida`	info	Se cambió una contraseña con código	`routes/auth.ts`
`regla_negocio`	warn	Cualquier rechazo con código: `CUPO_COMPLETO`, `FUERA_DE_HORARIO`, `YA_CANCELADO`, `SIN_PRECIO`, `NO_PAGABLE`, `CODIGO_*`	`middleware/errorHandler.ts`
`pago_aprobado`	info	Reserva pagada y confirmada	`lib/pagos.ts`
`pago_devuelto`	info / warn	Devolución por cancelación o por pago no esperado	`routes/pedidos.ts`, `lib/pagos.ts`
`mercadopago_sin_configurar` / `_sin_conexion` / `_error`	error	Falla de Mercado Pago	`lib/mercadoPago.ts`
`correo_sin_configurar` / `correo_fallido`	error	Falla del correo	`lib/mailer.ts`
`error_servidor`	error	Error no controlado (500)	`middleware/errorHandler.ts`

Para qué sirve: si un cliente dice “no pude pedir”, se busca en Render Logs a esa hora el evento `regla_negocio` y se ve si fue por `CUPO_COMPLETO`, por `FUERA_DE_HORARIO` o por un `error_servidor`.

Qué nunca se registra: el encabezado `Authorization`, las cookies, las contraseñas ni los hashes (`redact` en `lib/logger.ts`).

5. Salud y seguridad de la API

- `GET /api/health` → `200 { estado: "ok", servicio, hora }`. Lo usa Render para saber si la API está viva y sirve para "despertarla".
- `helmet` agrega encabezados de seguridad HTTP.
- CORS sólo acepta los orígenes de `CORS_ORIGIN` (en producción, la URL del frontend) y los métodos y encabezados que usa la app.
- El cuerpo JSON tiene un límite de 100 kB.
- Ningún secreto está en el código: todos salen de `config/env.ts`.

6. Configuración

Variable	Obligatoria	Uso
`DATABASE_URL`	sí	Conexión a PostgreSQL
`JWT_SECRET`	sí	Firma de tokens y HMAC de códigos de recuperación
`PORT`, `NODE_ENV`, `CORS_ORIGIN`, `LOG_LEVEL`	no	Puerto, entorno, orígenes permitidos, nivel de log
`SMTP_HOST`, `SMTP_PORT`, `SMTP_USER`, `SMTP_PASS`, `MAIL_FROM`	no (sin ellas, recuperar contraseña da 503)	Correo
`MP_ACCESS_TOKEN`, `FRONTEND_URL`, `API_URL`	no (sin token, reservar da 503)	Mercado Pago
`VITE_API_URL` (frontend)	no (por defecto `http://localhost:3000`)	URL de la API, se fija al construir

7. Tests

`npm test` (en `backend/`) corre **78 tests**: 20 unitarios y 58 de integración, y genera la cobertura.

Archivo	Tipo	Tests
`tests/unit/reglasPedido.test.ts`	unitario	19
`tests/unit/mailer.test.ts`	unitario	1
`tests/integration/auth.test.ts`	integración	11
`tests/integration/recuperacion.test.ts`	integración	7
`tests/integration/menus.test.ts`	integración	10
`tests/integration/pedidos.test.ts`	integración	11
`tests/integration/pagos.test.ts`	integración	14
`tests/integration/health.test.ts`	integración	5 (salud, CORS, 404 y JSON mal formado)

- Los de integración hacen la petición HTTP completa con `supertest` contra una **base de pruebas separada** (`localhost:5433/viandas_test`), nunca la de desarrollo ni la de producción. Cada archivo limpia sus tablas y los archivos corren de a uno.
- Mercado Pago se simula reemplazando `fetch`, y el correo con `vi.mock`.

- **Cobertura:** si las líneas cubiertas bajan del **65 %**, `npm test` falla. Sólo se excluyen `src/server.ts` (punto de entrada) y `src/config/**` (lectura de variables). El informe queda en `backend/coverage/index.html`.

8. Pipeline de CI/CD

Se dispara con cada `push` y cada pull request a `main` (rama protegida: los cambios entran por PR).

Job	Pasos	Cuándo corre
`backend`	`npm ci` → `prisma generate` → `prisma migrate deploy` sobre una base PostgreSQL efímera → `npm test` (con umbral 65 %) → sube la cobertura como artefacto → `npm run build`	push y PR
`frontend`	`npm ci` → `npm run build`	push y PR
`desplegar`	`curl` a los *deploy hooks* de Render del backend y del frontend	sólo en push a `main` y sólo si `backend` y `frontend` pasaron

Si falla un test o baja la cobertura, `desplegar` no corre y producción no cambia. El recorrido completo, con los secretos, está en el documento de arquitectura y despliegue.

9. Para modificarlo

Si piden…	Tocar
Un manejo de error que no existe	lanzar `new HttpError(status, 'mensaje', 'CODIGO')` donde ocurre; el manejador central ya lo responde y lo registra
Un evento nuevo en los logs	`logger.info({ evento: 'nombre', ...datos }, 'mensaje')`
Una variable de entorno nueva	`config/env.ts`, `.env.example`, README y el panel de Render